

# Passos VS Code

1. `git config --global --add safe.directory '*'`
2. Flutter: New Project
3. Manage: Trust
4. Finalizar tarefa: Git for windows

# Armazenando dados

É possível usar `shared_preferences` para armazenar dados simples no formato:

```
chave=>valor
```

# Tipos de dados

shared\_preferences aceita os seguintes tipos de dados:

- int
- double
- bool
- String
- List<String>

# Instalação

1. Ative o modo desenvolvedor:

```
start ms-settings:developers
```

2. Instale o pacote:

```
flutter pub add shared_preferences
```

# Salvar dados

```
final prefs = await SharedPreferences.getInstance();  
  
// salva chave 'x', valor 3  
await prefs.setInt('x', 3);
```

# Prática

Altere `aula10_codigo2.dart` para salvar o número digitado pelo usuário.

# Solução

aula10\_codigo3.dart

# Ler dados

```
final prefs = await SharedPreferences.getInstance();  
final x = prefs.getInt('x') ?? 0;
```

# Prática

Altere o método `buscaX` de `aula10_codigo4.dart` para buscar e mostrar o valor de `x` salvo anteriormente.

# Solução

- `aula10_codigo5.dart`

# Apagar dados

```
final prefs = await SharedPreferences.getInstance();  
  
// remove x  
await prefs.remove('x');
```

# Prática

Programe o botão *Apagar* do código `aula10_codigo6.dart` para apagar o valor de `x`. Mostre `x` igual a `0` até o próximo salvamento.

# Solução

aula10\_codigo7.dart

# Prática

Ajuste o programa no arquivo **aula10\_codigo8.dart** para mudar e salvar o valor da variável **azul** conforme o usuário apertar o **Switch**.

# Solução

aula10\_codigo9.dart

# Prática

Crie uma aba de configuração de app. Receba o nome do usuário e o tema preferido (escuro ou claro). Use **TextField** para o nome. Use **Switch** para o tema. Salve usando **shared\_preferences**.

# Exercício

Receba o CEP do usuário e o número da casa, use a API viacep para carregar rua, bairro, cidade, estado. Salve os dados usando `shared_preferences`. Carregue os dados na próxima inicialização da APP. Permita que o usuário apague os dados salvos.

# Arquivos

É possível ler e escrever arquivos usando Flutter.

# path\_provider

O pacote `path_provider` fornece uma maneira de acessar arquivos independente da plataforma.

Locais possíveis: Pasta de documentos e pasta temporária.

# Acessando documentos

```
import 'package:path_provider/path_provider.dart';

// ...
Future<String> get _pastaDocumentos async {
  final directory = await getApplicationDocumentsDirectory();

  return directory.path;
}
```

# Referência

A classe `File` cria uma referência para um arquivo.

```
Future<File> get _arquivo async {  
    final caminho = await _pastaDocumentos;  
    return File('$caminho/usuarios.txt');  
}
```

# Escrever no arquivo

Para escrever String no arquivo use o método:

```
writeAsString() .
```

```
Future<File> escreveX(int x) async {  
    final arquivo = await _arquivo;  
  
    return arquivo.writeAsString('$x');  
}
```

# Ler do arquivo

```
Future<int> lerX() async {  
  try {  
    final arquivo = await _arquivo;  
  
    final conteudo = await arquivo.readAsString();  
  
    return int.parse(conteudo);  
  } catch (_) {  
    // se acontecer um erro  
    return 0;  
  }  
}
```

# Prática

Crie um app que permita salvar linhas de texto em um arquivo chamado diário.txt. Cada nova linha deve começar com a data. Em seguida o texto que foi digitado pelo usuário. Crie uma janela para visualizar todo o diário.

# Exercício

Crie uma app que permite editar um arquivo Markdown chamado `organiza.md`. Na app o usuário deve poder escrever o texto, salvar o texto e apagar.

# Envio

- <https://forms.gle/tY6fmJBUo2yXm6GG8>

